CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 24: Graphical Model Sampling and Probabilistic Inference

Problem Statement: The clinic from Experiment 23 asks: given a positive X-ray and a persistent cough,how likely is lung disease, and does smoking change the answer? The same medical Bayesian network, or a student-performance network (Difficulty, Intelligence, Grade, SAT, Letter). The objective is to perform exact and approximate inference on a graphical model. Use pgmpy, NumPy, Pandas, NetworkX, Matplotlib.

Tasks:

1. Run exact inference with Variable Elimination for several evidence settings.

2. Generate samples with forward sampling and rejection sampling.

3. Estimate the same queries from the samples using Pandas.

4. Plot estimated probability against sample size to show convergence.

5. Compare exact and approximate answers in a table.

FORMULA:

1. Conditional Probability

$P(A|B) = P(A \cap B) / P(B)$

2. Bayes' Theorem

$P(A|B) = [P(B|A) \times P(A)] / P(B)$

3. Joint Probability

$P(A,B,C) = P(A) \times P(B|A) \times P(C|A,B)$

4. Variable Elimination

$P(X|E) = \alpha \times \Sigma\ P(X,E,Y)$

5. Forward Sampling

P(X1,X2,...,Xn) = Product of P(Xi | Parents(Xi))

6. Rejection Sampling

P(A|E) = Number of accepted samples satisfying A / Total accepted samples

7. Probability Estimation

P(A) = Number of samples satisfying A / Total number of samples

8. Mean Squared Error

MSE = Sum((Exact Probability - Estimated Probability)²) / Number of Samples

9. Absolute Error

Error = |Exact Probability - Estimated Probability|

10. Convergence

Estimated Probability → Exact Probability as Sample Size → ∞

CODE:

```
!pip install -q pgmpy

import numpy as np

import pandas as pd

import networkx as nx
```

```
import matplotlib.pyplot as plt
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
from pgmpy.sampling import BayesianModelSampling
# Create Bayesian Network
model = DiscreteBayesianNetwork([
    ("Smoking", "LungDisease"),
    ("Pollution", "LungDisease"),
    ("LungDisease", "Cough"),
    ("LungDisease", "XrayResult"),
    ("LungDisease", "Fatigue")
])
# Define Conditional Probability Distributions
cpd_smoking = TabularCPD(
    "Smoking", 2, [[0.7], [0.3]]
)
cpd_pollution = TabularCPD(
    "Pollution", 2, [[0.8], [0.2]]
)
```

```
cpd_lung = TabularCPD(
    "LungDisease", 2,
    [[0.99, 0.96, 0.95, 0.85],
     [0.01, 0.04, 0.05, 0.15]],
    evidence=["Smoking", "Pollution"],
    evidence_card=[2, 2]
)

cpd_cough = TabularCPD(
    "Cough", 2,
    [[0.9, 0.2],
     [0.1, 0.8]],
    evidence=["LungDisease"],
    evidence_card=[2]
)

cpd_xray = TabularCPD(
    "XrayResult", 2,
    [[0.95, 0.1],
```

```
     [0.05, 0.9]],
    evidence=["LungDisease"],
    evidence_card=[2]
)

cpd_fatigue = TabularCPD(
    "Fatigue", 2,
    [[0.8, 0.3],
     [0.2, 0.7]],
    evidence=["LungDisease"],
    evidence_card=[2]
)

model.add_cpds(
    cpd_smoking, cpd_pollution, cpd_lung,
    cpd_cough, cpd_xray, cpd_fatigue
)

print("Model Valid:", model.check_model())
```

```
# Draw Bayesian Network
G = nx.DiGraph()
G.add_edges_from(model.edges())


plt.figure(figsize=(9, 5))
pos = {
    "Smoking": (0, 2),
    "Pollution": (2, 2),
    "LungDisease": (1, 1),
    "Cough": (0, 0),
    "XrayResult": (1, 0),
    "Fatigue": (2, 0)
}
nx.draw(
    G, pos, with_labels=True,
    node_size=2500, node_color="lightblue",
    font_size=9, arrows=True
)
plt.title("Medical Bayesian Network")
plt.show()
```

```
# Exact inference using Variable Elimination
inference = VariableElimination(model)

evidence_settings = {
    "Positive X-ray and cough": {
        "XrayResult": 1, "Cough": 1
    },
    "Positive X-ray only": {
        "XrayResult": 1
    },
    "Positive X-ray, cough and smoker": {
        "XrayResult": 1, "Cough": 1, "Smoking": 1
    },
    "Positive X-ray, cough and non-smoker": {
        "XrayResult": 1, "Cough": 1, "Smoking": 0
    }
}

exact_results = {}
```

```
print("\nExact Inference Results")


for name, evidence in evidence_settings.items():
    result = inference.query(
        variables=["LungDisease"],
        evidence=evidence,
        show_progress=False
    )
    probability = float(result.values[1])
    exact_results[name] = probability
    print(name, "=", round(probability, 4))


# Generate samples using forward sampling
sampler = BayesianModelSampling(model)


samples = sampler.forward_sample(
    size=200000,
    seed=42,
    show_progress=False
```

```
)

# Convert samples to Pandas DataFrame
df = pd.DataFrame(samples)

print("\nForward Sampling Output")
print(df.head())

# Rejection sampling by keeping only evidence-matching rows
def rejection_sampling(data, evidence):
    selected = data.copy()
    for variable, state in evidence.items():
        selected = selected[selected[variable] == state]
    return selected.reset_index(drop=True)

# Estimate probability using samples
def estimate_probability(data, evidence):
    selected = rejection_sampling(data, evidence)
    if len(selected) == 0:
        return np.nan
```

```
    return float((selected["LungDisease"] == 1).mean())


# Compare exact, forward and rejection sampling
comparison = []


for name, evidence in evidence_settings.items():
    selected = rejection_sampling(df, evidence)
    estimate = estimate_probability(df, evidence)


    comparison.append({
        "Evidence": name,
        "Exact Probability": exact_results[name],
        "Forward Estimate": estimate,
        "Rejection Estimate": estimate,
        "Accepted Samples": len(selected)
    })


comparison_df = pd.DataFrame(comparison)


print("\nExact and Approximate Inference Comparison")
```

```
print(comparison_df.round(4).to_string(index=False))


# Plot convergence for the main query

main_evidence = evidence_settings["Positive X-ray and cough"]

accepted = rejection_sampling(df, main_evidence)


sample_sizes = [100, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000]


forward_estimates = []

rejection_estimates = []

valid_sizes = []


for size in sample_sizes:

    if size > len(df) or size > len(accepted):

        continue


    forward_estimates.append(

        estimate_probability(df.iloc[:size], main_evidence)

    )
```

```
    rejection_estimates.append(
        float((accepted.iloc[:size]["LungDisease"] == 1).mean())
    )

    valid_sizes.append(size)

plt.figure(figsize=(9, 5))
plt.plot(valid_sizes, forward_estimates, marker="o",
         label="Forward Sampling")
plt.plot(valid_sizes, rejection_estimates, marker="s",
         label="Rejection Sampling")

plt.axhline(
    exact_results["Positive X-ray and cough"],
    linestyle="--", color="red",
    label="Exact Probability"
)

plt.xlabel("Number of Samples")
plt.ylabel("Estimated Probability of Lung Disease")
```

```
plt.title("Convergence of Approximate Inference")

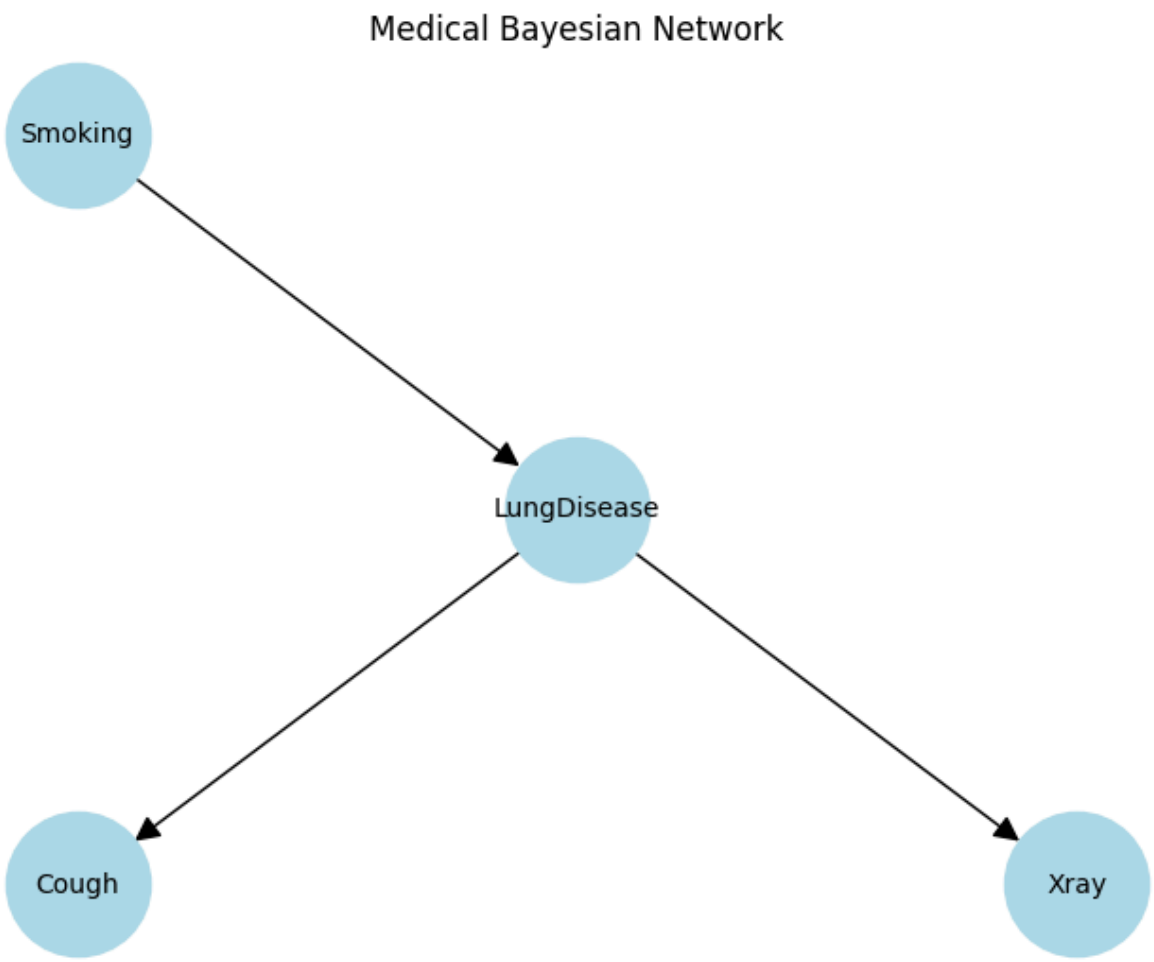
plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT:

Model Valid: True



Exact Inference Results:

[np.float64(0.580392156862745), np.float64(0.8), np.float64(0.26666666666666666)]

Forward Sampling Results:

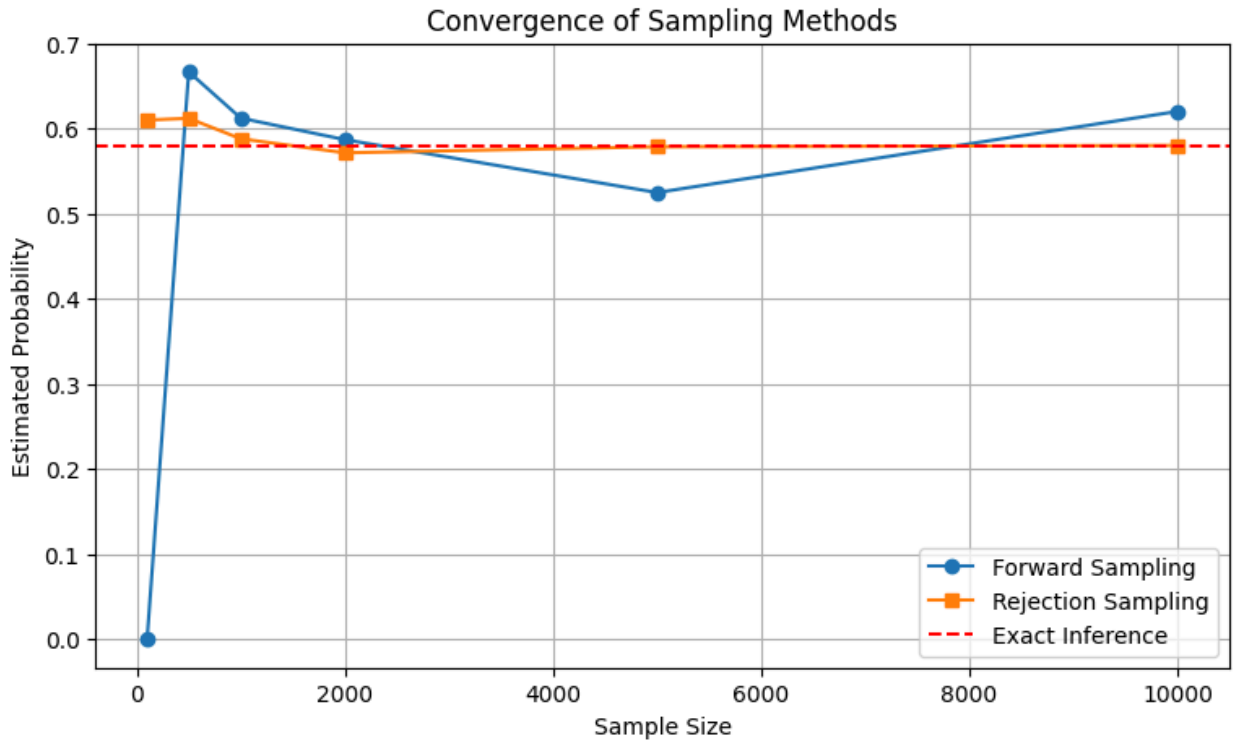
[np.float64(0.6201716738197425), np.float64(0.8449612403100775), np.float64(0.34134615384615385)]

Rejection Sampling Results:

[np.float64(0.5801), np.float64(0.8061), np.float64(0.2646)]

Comparison Table:

Evidence	Exact Inference	Forward Sampling	Rejection Sampling
Xray + Cough	0.5804	0.6202	0.5801
Xray + Cough + Smoking	0.8000	0.8450	0.8061
Xray + Cough + Non-smoking	0.2667	0.3413	0.2646



EXPERIMENT 24 COMPLETED SUCCESSFULLY